



# **Universidad Nacional Autónoma de México.**



Facultad de Ingeniería.

## **Sistemas Distribuidos.**

*Profesor: Ing. Óscar Manuel Ruíz Hurtado.*

*Semestre 2027-1*

Laboratorio S5.

Mexi Banco: de un monolito a servicios que se encuentran solos.

Grupo: 2

**Alumna:**

**Valencia Lerdo Dalia Jimena**

## Paso 1. La historia.

Al pasar la llamada a método HTTP se rompió la atomicidad de la transferencia. En el monolito, transferir llamaba a cuentas.cargar y cuentas.abonar dentro de un solo @Transactional y con una sola base de datos. Si el abono fallaba, la base deshacía también el cargo por lo que la operación se hacía o no se hacía.

Al separar el banco en servicios, cuentas dejó de ser un objeto en memoria y pasó a ser un cliente que hace una petición HTTP a otro proceso, con su propia base. Cuando cuenta confirma el cargo, ese cargo ya quedó guardado en su base y transferencia no puede deshacerlo. Si el abono falla después, el dinero queda cargado en el origen y nunca llega al destino. Cada servicio confirma solo lo suyo y ya no hay una transacción que cubra a todos.

Además aparecieron tres problemas más: la dirección de cuenta está escrita a mano en el compose (CUENTA\_URL: http://cuenta:8080), cada servicio tiene su propia puerta abierta a cualquiera, y una dirección fija solo apunta a una copia, así que no se puede repartir si cuenta tiene varias. De ahí salen los pasos del laboratorio: directorio, gateway y balanceo.

## Paso 2. Directorio.

En un directorio (service discovery) cada servicio se anota al arrancar con su nombre y su IP, manda un heartbeat cada 5 s y, si el directorio deja de recibirlo durante 15 s, lo borra. Usé Eureka, el directorio de Spring Cloud.

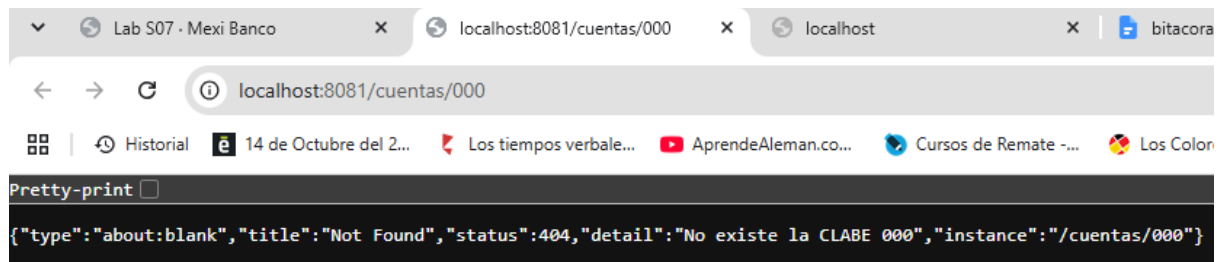
Con el proyecto tal como se construyó, docker compose ps mostraba seis contenedores, cada uno con su propio puerto del 8081 al 8085, y no existía ningún contenedor llamado discovery. Cuenta respondía directo en localhost:8081 con un 404 en JSON ("No existe la CLABE 000"), mientras que localhost:8761 no cargaba porque todavía no había directorio.

**Imagen 1.** docker compose up -d y docker compose ps: seis contenedores, sin discovery.

```
C:\Users\Jimena\Documents\S07\mexi-banco>docker compose up -d
[+] up 7/7
Network mexibanco-v06 Created
0.1s Container mexi-banco-v06-db-1 Started
7.5s Container mexi-banco-v06-notificacion-1 Started
6.8s Container mexi-banco-v06-transferencia-1 Started
7.6s Container mexi-banco-v06-cuenta-1 Started
7.6s Container mexi-banco-v06-movimiento-1 Started
7.7s Container mexi-banco-v06-spei-1 Started

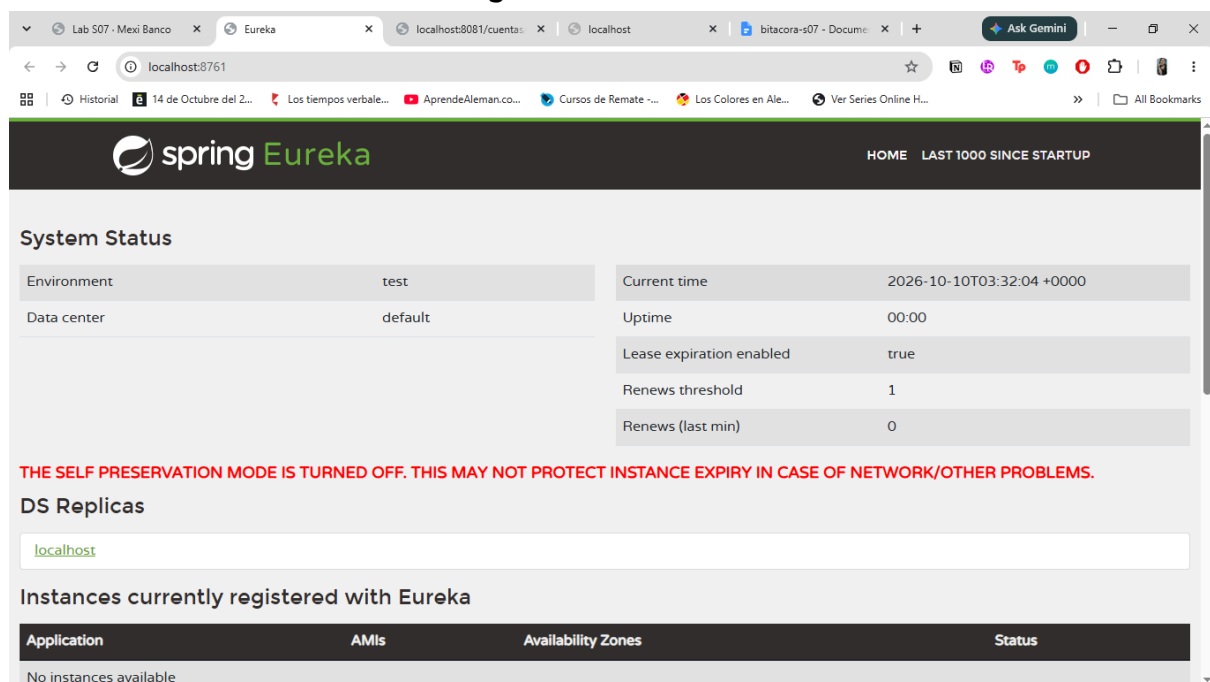
C:\Users\Jimena\Documents\S07\mexi-banco>docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS                               PORTS
mexi-banco-v06-cuenta-1             mexi-banco-v06-cuenta              "java -jar app.jar"                  cuenta     12 seconds ago Up 5 seconds (health: starting)    0.0.0.0:8081
mexi-banco-v06-db-1                 postgres:17-alpine                 "docker-entrypoint.s..."            db         12 seconds ago Up 11 seconds (healthy)           5432/tcp
mexi-banco-v06-movimiento-1         mexi-banco-v06-movimiento          "java -jar app.jar"                  movimiento 12 seconds ago Up 5 seconds (health: starting)    0.0.0.0:8082
mexi-banco-v06-notificacion-1       mexi-banco-v06-notificacion        "java -jar app.jar"                  notificacion 12 seconds ago Up 5 seconds (health: starting)    0.0.0.0:8084
mexi-banco-v06-spei-1               mexi-banco-v06-spei                "java -jar app.jar"                  spei       12 seconds ago Up 5 seconds (health: starting)    0.0.0.0:8085
mexi-banco-v06-transferencia-1      mexi-banco-v06-transferencia       "java -jar app.jar"                  transferencia 12 seconds ago Up 5 seconds (health: starting)    0.0.0.0:8083
```

Imagen 2. localhost:8081/cuentas/000: cuenta responde directo con 404 en JSON.



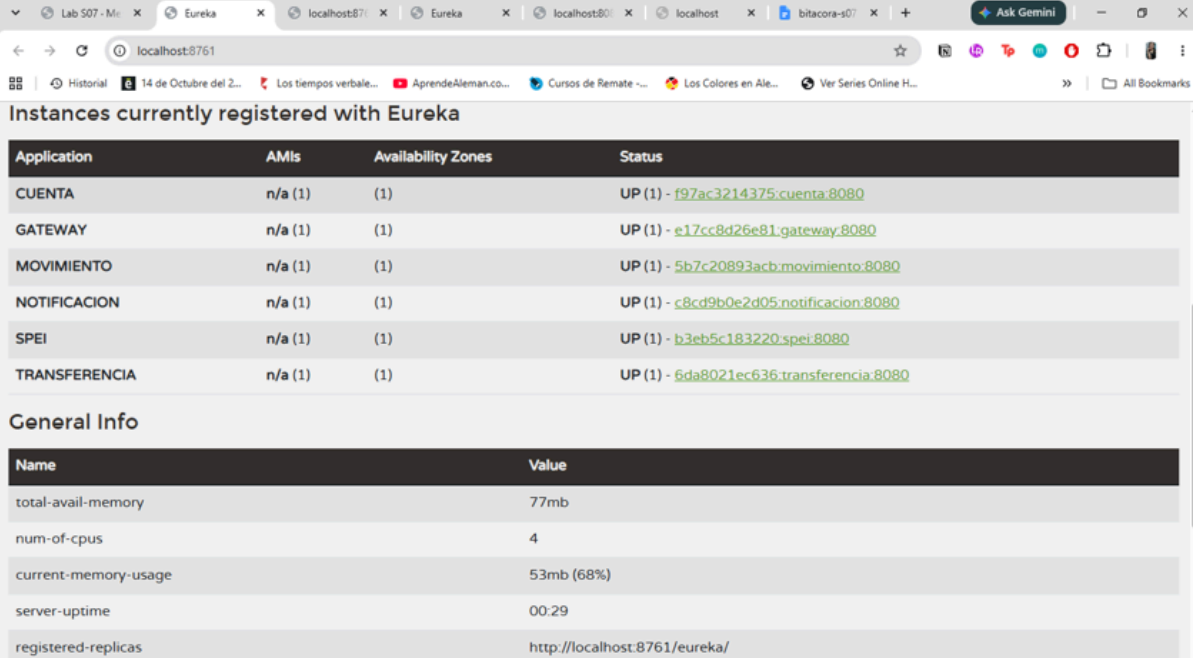
Después de activar `@EnableEurekaServer`, las dos propiedades del directorio, la dependencia del cliente en los cinco `pom.xml` y el servicio `discovery` en el `compose`, el tablero de Eureka ya es visible. Recién arrancado, no tenía instancias registradas.

Imagen 3. localhost:8761



Unos segundos después, los servicios se anotaron solos. Aparecen CUENTA, MOVIMIENTO, NOTIFICACION, SPEI y TRANSFERENCIA, cada uno con UP (1), y también GATEWAY, que se anota igual que los demás porque también es cliente de Eureka.

Imagen 4. Tablero de Eureka con los servicios registrados



The screenshot shows the Eureka dashboard in a web browser. The top section, 'Instances currently registered with Eureka', contains a table with the following data:

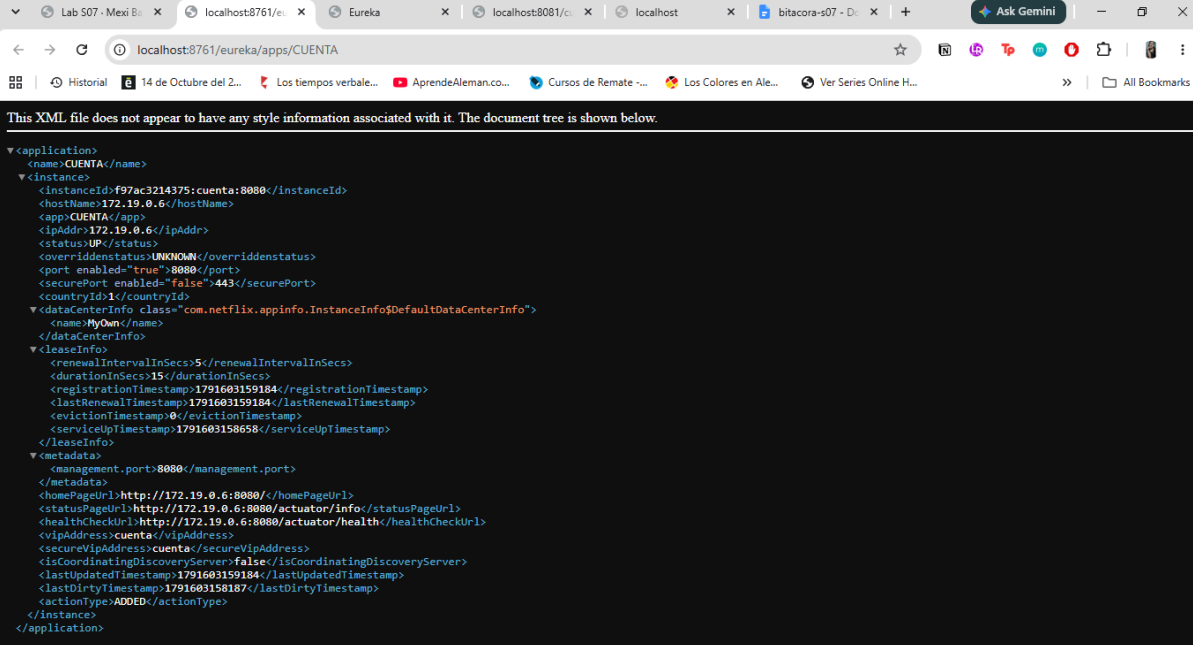
| Application   | AMIs    | Availability Zones | Status                                                   |
|---------------|---------|--------------------|----------------------------------------------------------|
| CUENTA        | n/a (1) | (1)                | UP (1) - <a href="#">f97ac3214375:cuenta:8080</a>        |
| GATEWAY       | n/a (1) | (1)                | UP (1) - <a href="#">e17cc8d26e81:gateway:8080</a>       |
| MOVIMIENTO    | n/a (1) | (1)                | UP (1) - <a href="#">5b7c20893acb:movimiento:8080</a>    |
| NOTIFICACION  | n/a (1) | (1)                | UP (1) - <a href="#">c8cd9b0e2d05:notificacion:8080</a>  |
| SPEI          | n/a (1) | (1)                | UP (1) - <a href="#">b3eb5c183220:spei:8080</a>          |
| TRANSFERENCIA | n/a (1) | (1)                | UP (1) - <a href="#">6da8021ec636:transferencia:8080</a> |

The bottom section, 'General Info', contains a table with the following data:

| Name                 | Value                                                                     |
|----------------------|---------------------------------------------------------------------------|
| total-avail-memory   | 77mb                                                                      |
| num-of-cpus          | 4                                                                         |
| current-memory-usage | 53mb (68%)                                                                |
| server-uptime        | 00:29                                                                     |
| registered-replicas  | <a href="http://localhost:8761/eureka/">http://localhost:8761/eureka/</a> |

La misma información que muestra el tablero, pero para que la consulten los servicios, está en `localhost:8761/eureka/apps/CUENTA`. Ahí vi: `hostName` y `ipAddr` 172.19.0.6, port 8080, status UP, `renewalIntervalInSecs` 5 y `durationInSecs` 15, que son el heartbeat y el plazo del compose.

Imagen 5. `localhost:8761/eureka/apps/CUENTA`: XML



The screenshot shows the XML response from the Eureka API. The XML structure is as follows:

```
<?xml version="1.0"?>
<application>
  <name>CUENTA</name>
  <instance>
    <instanceId>f97ac3214375:cuenta:8080</instanceId>
    <hostName>172.19.0.6</hostName>
    <app>CUENTA</app>
    <ipAddr>172.19.0.6</ipAddr>
    <status>UP</status>
    <overrideStatus>UNKNOWN</overrideStatus>
    <port enabled="true">8080</port>
    <securePort enabled="false">443</securePort>
    <countryId>1</countryId>
    <dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
      <name>MyOwn</name>
    </dataCenterInfo>
    <leaseInfo>
      <renewalIntervalInSecs>5</renewalIntervalInSecs>
      <durationInSecs>15</durationInSecs>
      <registrationTimestamp>1791603159184</registrationTimestamp>
      <lastRenewalTimestamp>1791603159184</lastRenewalTimestamp>
      <evictionTimestamp>0</evictionTimestamp>
      <serviceUpTimestamp>1791603158658</serviceUpTimestamp>
    </leaseInfo>
    <metadata>
      <management.port>8080</management.port>
    </metadata>
    <homePageUrl>http://172.19.0.6:8080/</homePageUrl>
    <statusPageUrl>http://172.19.0.6:8080/actuator/info</statusPageUrl>
    <healthCheckUrl>http://172.19.0.6:8080/actuator/health</healthCheckUrl>
    <vipAddress>cuenta</vipAddress>
    <secureVipAddress>cuenta</secureVipAddress>
    <isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
    <lastUpdatedTimestamp>1791603159184</lastUpdatedTimestamp>
    <lastDirtyTimestamp>1791603158187</lastDirtyTimestamp>
    <actionType>ADDED</actionType>
  </instance>
</application>
```

¿Alguien usa el directorio para encontrar a cuenta?

Para este momento no. Aunque los cinco servicios ya están anotados, transferencia sigue llamando a la dirección escrita a mano, CUENTA\_URL: http://cuenta:8080, y ese nombre lo resuelve el DNS de Docker, no Eureka. El directorio ya sabe dónde está cuenta, pero nadie le pregunta. Quien sí lo usa en este punto es el gateway, con lb://cuenta. Esto lo resolví en el paso 4 y en la tarea.

### Paso 3. Gateway.

Un API gateway es la única entrada para los clientes (puerto 8080). Según la ruta de cada petición decide a qué servicio mandarla, y lo que no tiene regla no pasa. Escribí las cinco rutas en gateway/src/main/resources/application.yml con lb://servicio, así que el gateway le pregunta al directorio dónde está cada uno. Movimiento y notificación solo aceptan GET y SPEI solo acepta POST.

Registrar un movimiento es una operación interna, pero llamando directo al puerto 8082 responde 400 ("faltan datos") y no 404: el endpoint existe y está abierto para cualquiera.

**Imagen 6.** curl -i -X POST localhost:8082/movimientos

```
C:\Users\Jimena\Documents\S07\mexi-banco>curl -i -X POST http://localhost:8082/movimientos
HTTP/1.1 400
Content-Type: application/problem+json
Transfer-Encoding: chunked
Date: Sat, 10 Oct 2026 03:36:43 GMT
Connection: close

{"type":"about:blank","title":"Bad Request","status":400,"detail":"Failed to read request","instance":"/movimientos"}
```

**Imagen 7.** Dos 404 por el gateway: el primero con X-Instancia, el segundo sin ella.

```
C:\Users\Jimena\Documents\S07\mexi-banco>curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: f97ac3214375
Content-Type: application/problem+json
Date: Sat, 10 Oct 2026 04:01:12 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}
C:\Users\Jimena\Documents\S07\mexi-banco>curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 130

{"timestamp":"2026-10-10T04:01:22.746+00:00","path":"/api/movimientos","status":404,"error":"Not Found","requestId":"8156708b-23"}
C:\Users\Jimena\Documents\S07\mexi-banco>
```

Los dos dieron 404, pero no vienen del mismo lugar.

GET /api/cuentas/000: trae X-Instancia (f97ac3214375), o sea que la petición sí pasó por el gateway, él le preguntó al directorio dónde estaba cuenta y llegó hasta allá. El 404 lo dio la propia cuenta, con su mensaje "No existe la CLABE 000".

POST /api/movimientos: no trae X-Instancia y el error tiene el formato del gateway, no el de un servicio. Como ninguna ruta deja pasar un POST a movimientos, el gateway lo cortó en la puerta y nunca llegó a movimiento. En contraste con el 400 del puerto 8082, donde sí llegaba, desde afuera esa operación interna ya no se alcanza.

Creé una cuenta por el gateway con una CLABE única (S071791605370). El alta respondió 201 Created y la consulta 200 OK con saldo 0.00; ambas traen X-Instancia, así que las atendió cuenta.

Imagen 8 · Alta con 201, consulta con 200, saldo 0.00 y la liga de la cuenta.

```
Jimena@DESKTOP-ECC079K MINGW64 ~/Documents/S07/mexi-banco (trabajo-s07)
$ LAB_CLABE="S07$(date +%s)"
curl -i -X POST http://localhost:8080/api/cuentas \
  -H 'Content-Type: application/json' \
  -d '{"clabe":"$LAB_CLABE","titular":"S07","saldoInicial":0}'

URL="http://localhost:8080/api/cuentas/$LAB_CLABE" # guardamos la dirección de tu cuenta
curl -i "$URL"
echo "$URL"
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: f97ac3214375
Content-Type: application/json
Date: Sat, 10 Oct 2026 04:09:31 GMT

HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: f97ac3214375
Content-Type: application/json
Date: Sat, 10 Oct 2026 04:09:32 GMT

{"id":1,"clabe":"S071791605370","titular":"S07","saldo":0.00}http://localhost:8080/api/cuentas/S071791605370
```

Pasos 4 y 5. Copias y caída.

Escalar horizontalmente es levantar varias copias idénticas de un servicio y repartir las peticiones entre ellas. En el compose cambié los ports de cuenta por deploy: replicas: 3, y quité los puertos publicados de movimiento, transferencia, notificación y SPEI; solo el gateway (8080) y el directorio (8761) quedan abiertos hacia afuera.

Los tres IDs de mis copias son 3b525d47341f (cuenta-1), e41eb0b9c0e3 (cuenta-2) y 1e5ba1d269fc (cuenta-3). Cada una escucha por dentro en 8080/tcp, sin puerto publicado en mi laptop.

Imagen 9. docker ps: tres contenedores de cuenta, con 8080/tcp sin flecha.

```
Jimena@DESKTOP-ECC079K MINGW64 ~/Documents/S07/mexi-banco (trabajo-s07)
$ docker ps --filter name=cuenta --format '{{.ID}} {{.Names}} {{.Ports}}'
e41eb0b9c0e3 mexi-banco-v06-cuenta-2 8080/tcp
1e5ba1d269fc mexi-banco-v06-cuenta-3 8080/tcp
3b525d47341f mexi-banco-v06-cuenta-1 8080/tcp
```

Imagen 10. Tablero de Eureka: CUENTA con UP (3) y los mismos tres IDs

| Application | Availability |       |                                                                                                                                               |
|-------------|--------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------|
|             | AMIs         | Zones | Status                                                                                                                                        |
| CUENTA      | n/a<br>(3)   | (3)   | UP (3) - <a href="#">1e5ba1d269fc:cuenta:8080</a> ,<br><a href="#">3b525d47341f:cuenta:8080</a> ,<br><a href="#">e41eb0b9c0e3:cuenta:8080</a> |

Con las nueve consultas a mi cuenta, la cabecera X-Instancia se fue turnando entre los tres IDs en el mismo orden (e41eb0b9c0e3, 1e5ba1d269fc, 3b525d47341f), tres veces cada una. Es un reparto por turnos del tipo round robin.

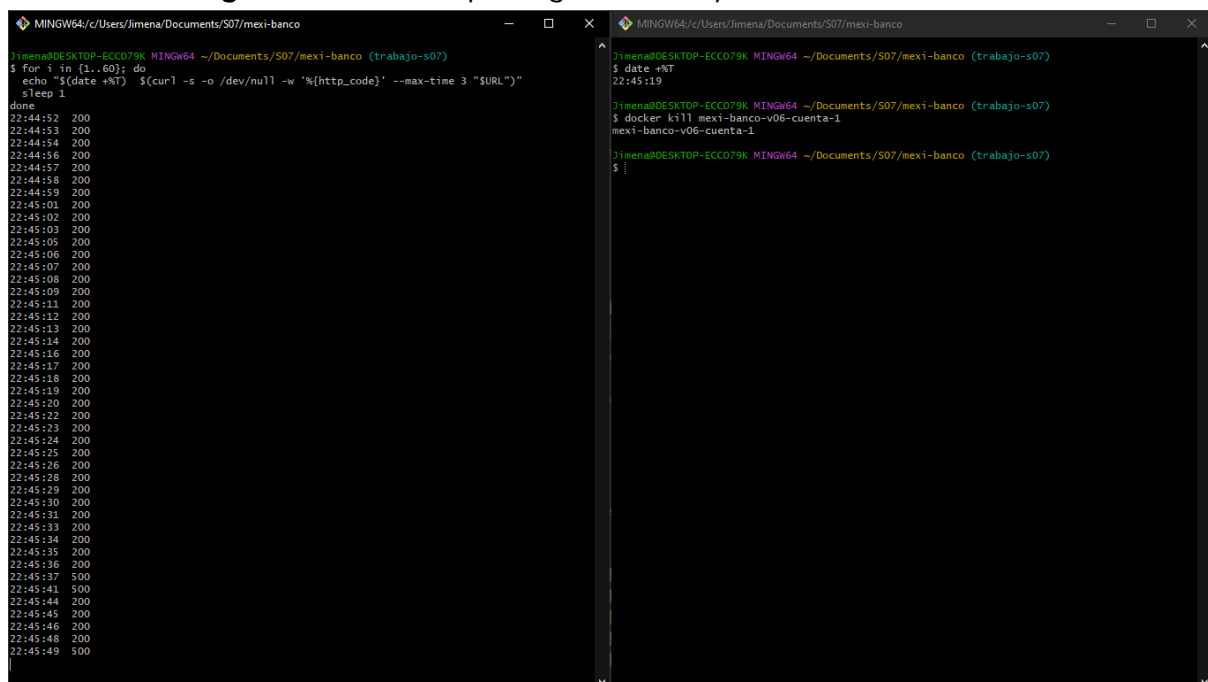
**Imagen 11.** Nueve consultas: X-Instancia rota entre las tres copias.

```
jimena@DESKTOP-ECC079K MINGW64 ~/Documents/S07/mexi-banco (trabajo-s07)
$ for i in {1..9}; do
  curl -sS -D - -o /dev/null "$URL" | grep -i '^X-Instancia'
done
X-Instancia: e41eb0b9c0e3
X-Instancia: 1e5ba1d269fc
X-Instancia: 3b525d47341f
X-Instancia: e41eb0b9c0e3
X-Instancia: 1e5ba1d269fc
X-Instancia: 3b525d47341f
X-Instancia: e41eb0b9c0e3
X-Instancia: 1e5ba1d269fc
X-Instancia: 3b525d47341f
```

¿Transferencia también reparte? En ese punto no. El reparto con directorio solo lo hacía el gateway. Por dentro, transferencia llamaba a cuenta con CUENTA\_URL: http://cuenta:8080, un nombre que traduce el DNS de Docker y no Eureka, así que no usaba el directorio ni revisaba qué copias estaban sanas. Esto se corrige en la sección tarea, con @LoadBalanced y las URLs solo por nombre.

Usé dos terminales: en una, una consulta por segundo a mi cuenta con la hora y el código de respuesta; en la otra, docker kill mexi-banco-v06-cuenta-1, que corta la copia de golpe.

**Imagen 12.** Consultas por segundo. date y docker kill de cuenta-1.



```
MINGW64/c/Users/jimena/Documents/S07/mexi-banco
jimena@DESKTOP-ECC079K MINGW64 ~/Documents/S07/mexi-banco (trabajo-s07)
$ for i in {1..60}; do
  echo "$(date +%T) $(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL")"
  sleep 1
done
22:44:52 200
22:44:53 200
22:44:54 200
22:44:56 200
22:44:57 200
22:44:58 200
22:44:59 200
22:45:01 200
22:45:02 200
22:45:03 200
22:45:05 200
22:45:06 200
22:45:07 200
22:45:08 200
22:45:09 200
22:45:11 200
22:45:12 200
22:45:13 200
22:45:14 200
22:45:16 200
22:45:17 200
22:45:18 200
22:45:19 200
22:45:20 200
22:45:22 200
22:45:23 200
22:45:24 200
22:45:25 200
22:45:26 200
22:45:28 200
22:45:29 200
22:45:30 200
22:45:31 200
22:45:33 200
22:45:34 200
22:45:35 200
22:45:36 200
22:45:37 500
22:45:41 500
22:45:44 200
22:45:45 200
22:45:46 200
22:45:48 200
22:45:49 500

MINGW64/c/Users/jimena/Documents/S07/mexi-banco
jimena@DESKTOP-ECC079K MINGW64 ~/Documents/S07/mexi-banco (trabajo-s07)
$ date +%T
22:45:19
jimena@DESKTOP-ECC079K MINGW64 ~/Documents/S07/mexi-banco (trabajo-s07)
$ docker kill mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1
jimena@DESKTOP-ECC079K MINGW64 ~/Documents/S07/mexi-banco (trabajo-s07)
$
```

**Imagen 13.** Tablero de Eureka con CUENTA en UP (2) a las 22:45:45: la copia 3b525d47341f ya no aparece.

| Application | Availability |       | Status                                                                                          |
|-------------|--------------|-------|-------------------------------------------------------------------------------------------------|
|             | AMIs         | Zones |                                                                                                 |
| CUENTA      | n/a<br>(2)   | (2)   | UP (2) - <a href="#">1e5ba1d269fc:cuenta:8080</a> ,<br><a href="#">e41eb0b9c0e3:cuenta:8080</a> |

Casi todas las respuestas fueron 200. Los 500 salieron salteados, estos son las peticiones a las que les tocó la copia muerta. Uno de ellos (22:45:49) salió incluso después de que la copia desapareció del tablero a las 22:45:45, porque el gateway sigue usando su propia copia de la lista durante unos segundos.

Hubo errores porque nadie le avisa al sistema que una copia murió; se va enterando por capas. El directorio solo se entera cuando dejan de llegar heartbeats y espera su plazo de 15 s para no borrar a alguien que solo se tardó. El gateway no ve el directorio en vivo: guarda una copia de la lista y la renueva cada 5 s, más 5 s de caché del balanceador. Mientras tanto, el gateway sigue mandando peticiones a una copia muerta. Con los valores de fábrica de Spring serían minutos.

Al final reviví la copia con docker start.

**Imagen 14.** docker start mexi-banco-v06-cuenta-1.

```
Jimena@DESKTOP-ECC079K MINGW64 ~/Documents/S07/mexi-banco (trabajo-s07)
$ docker start mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1
```

**Cierre.**

**El directorio** resuelve que las direcciones de los servicios no se escriban a mano por lo que cada servicio se anota solo con su nombre y su IP, manda un heartbeat para decir que sigue vivo y quien quiera llamarlo le pregunta al directorio.

**El gateway** resuelve que el cliente tuviera que conocer cinco puertas y que cualquiera pudiera llamar a operaciones internas: es una sola entrada, con reglas por ruta y por método, y lo que no tiene ruta (como POST /api/movimientos) da 404.

**El balanceador** resuelve el reparto de las peticiones entre las copias de un servicio, por turnos y dejar de mandar peticiones a las copias que ya no están en el directorio.

**Cuando murió una copia** vi 3 errores 500 salteados en 44 consultas, entre las 22:45:37 y las 22:45:49, porque el directorio y el gateway se enteran por capas y mientras tanto el gateway siguió mandando una de cada tres peticiones a la copia muerta.

Lo que prueba que transferencia ya reparte por dentro es la lista de quién atendió cada llamada: la consulta de la cuenta de origen la atendió cuenta-1 (3b525d47341f), la de destino cuenta-3 (1e5ba1d269fc), el cargo cuenta-2 (e41eb0b9c0e3) y el abono cuenta-1 (3b525d47341f). El cargo y el abono de la misma transferencia cayeron en copias distintas, porque cada llamada la reparte el balanceador de transferencia.